

How the Shift Operator Acts Relative to the Global Coin Operator in a Discrete-Time Quantum Walk

High-Level Picture

Short version: the **shift** doesn't actually depend on the **gigantic coin operator** at all. It only depends on the **graph wiring** (the PortMap / arc pairing). The “gigantic operator” you're thinking of is the **block-diagonal tensor of all local coins**, and the full one-step walk is

$$U = S \left(\bigoplus_v C_v \right),$$

where

- $\bigoplus_v C_v$ is the **huge block-diagonal coin operator** (tensor/direct sum of small local coins), and
- S is the **flip-flop shift**, a permutation that swaps each arc with its reverse.

So the idea is:

1. First you apply the big block-diagonal coin operator.
2. Then you apply the graph-defined shift S , which just permutes arc indices.

The shift knows nothing about the coin matrices themselves; it only knows the graph connectivity and the reverse-arc pairing.

1 Hilbert Space and the “Gigantic Coin Operator”

We work in the **arc basis**:

$$\mathcal{H} = \text{span}\{ |a\rangle : a \text{ is an oriented arc } (v \rightarrow w) \}.$$

For each vertex v , we have a local “coin subspace”

$$\mathcal{H}_v = \text{span}\{ |v, k\rangle : k = 1, \dots, d_v \},$$

where d_v is the degree of v . A local coin C_v is a $d_v \times d_v$ unitary acting only on $\mathcal{H}_v$.

If you stack all of them together, you get the **gigantic operator**

$$C_{\text{global}} = \bigoplus_v C_v.$$

In matrix form this is an $A \times A$ block-diagonal unitary, where

$$A = \sum_v d_v$$

is the total number of oriented arcs.

Conceptually, this is what a function like `apply_coin_blockdiag` does in code: it applies the right local block C_v to the state segment corresponding to arcs out of node v . Operationally:

- You group amplitudes by vertex v (all outgoing arcs from v).
- On each such group, you apply the local coin matrix C_v .
- Putting this together over all vertices gives the global block-diagonal action $\bigoplus_v C_v$.

2 What the Shift Operator Actually Is

The **shift operator** S is defined purely from the **pairing of each oriented arc with its reverse**:

- For each arc $a = (v \rightarrow w)$, there is a reverse arc $a^{\text{rev}} = (w \rightarrow v)$.
- In the arc basis, the shift acts as

$$S |a\rangle = |a^{\text{rev}}\rangle.$$

This means S is just a **permutation matrix**:

- there is exactly one 1 in each row and each column,
- $S^2 = I$ (it is an involution),
- $S^\dagger = S = S^{-1}$ (it is unitary and Hermitian).

In code-like terms:

- You maintain an array `rev[i]` which gives the index of the reverse arc of arc `i`.
- You define a permutation `perm` via `perm[i] = rev[i]`.
- Applying S is just reindexing the state vector:

$$(\psi_{\text{out}})_i = (\psi_{\text{in}})_{\text{perm}[i]}.$$

So the **action of the shift is completely independent of the coin matrices**. It only “knows” the graph and the flip–flop pairing (which arc is the reverse of which).

3 How They Combine: One Step = Coin Then Shift

One full discrete-time step of the walk is

$$\psi_{t+1} = S \left(\bigoplus_v C_v \right) \psi_t.$$

Operationally:

1. **Coin phase.** At each vertex v , we take the amplitudes on its outgoing arcs and apply the local unitary C_v . This is the gigantic block-diagonal operator $\bigoplus_v C_v$.
2. **Shift phase.** Then we **re-route those outgoing amplitudes to incoming arcs** of neighbors by applying the permutation S . In the flip-flop scheme: amplitude on arc $v \rightarrow w$ gets moved to arc $w \rightarrow v$.

In code, this is exactly what a composition like

```
psi_after_coin = apply_coin_blockdiag(psi, coins, portmap, ...)
psi_after_shift = shift.apply(psi_after_coin)
```

is doing.

Important point: S does *not* look inside the coin operator. The chronological order is:

$$\psi_t \xrightarrow{\bigoplus_v C_v} (\text{coin-updated amplitudes}) \xrightarrow{S} \psi_{t+1}.$$

The shift just permutes *whatever* amplitudes the coin stage produced on each arc.

4 Relation to the Tensor-Product Picture

Often, in the standard regular-graph formulation, we think in terms of a **position** $\otimes$ **coin** decomposition:

$$\mathcal{H} = \mathcal{H}_{\text{pos}} \otimes \mathcal{H}_{\text{coin}}.$$

For a d -regular graph with a homogeneous coin, we typically have:

- A single fixed coin C of dimension $d \times d$.
- A global coin $C_{\text{global}} = I_{\text{pos}} \otimes C$.

The shift is then a “conditional translation”:

$$S = \sum_{x,c} |x + f(c), c'\rangle \langle x, c|,$$

where $f(c)$ selects the neighbor according to coin state c , and c' is either the same or some updated direction label, depending on whether you use a moving or flip-flop shift.

In our **arc basis** formulation, we have essentially flattened both position and coin into a single index “arc,” so that:

- The **tensor product** / **direct sum of local coins** becomes the big block-diagonal operator $C_{\text{global}} = \bigoplus_v C_v$.
- The **conditional shift** becomes a pure permutation S dictated by the reverse-arc map.

So your “gigantic operator (tensor of small local coin operators)” is **only the coin part**. The shift *does not* act based on that gigantic operator; instead:

- First the gigantic operator changes amplitudes **within** each node’s arc block (local mixing).
- Then the shift **swaps whole arc basis vectors** according to the graph connectivity (routing).

Summary

- The global coin is a block-diagonal unitary $C_{\text{global}} = \bigoplus_v C_v$ acting locally at each vertex.
- The shift S is a fixed permutation determined solely by the graph's reverse-arc pairing, $S|a\rangle = |a^{\text{rev}}\rangle$.
- One time step of the walk is $U = SC_{\text{global}}$, i.e. coin then shift.
- The shift does not depend on the coin matrices themselves; it only depends on the graph wiring.